Laboratorio 6
Aprendizaje por Refuerzo

Institución: Universidad Complutense de Madrid
Grado: Ingeniería Informática
Curso: INTELIGENCIA ARTIFICIAL I - Grupo A
Laboratorio realizado por: Laura Valles Ramírez y Diego Barba Arévalo
Fecha de entrega: 11/12/2025

ÍNDICE

1. Introducción **2**
2. Descripción del Entorno **2**
2.1. Elementos 2
2.2. Dinámica de la Receta 3
3. Definición del Problema **3**
3.1. Espacio de Acciones 3
3.2. Espacio de Estados 4
4. Sistema de Recompensas **4**
4.1. Penalizaciones 4
4.2. Recompensas 4
4.3. Mecánica de Cocción 5
5. Implementación del Algoritmo Q-Learning **5**
5.1. Hiperparámetros utilizados 5
6. Visualización y Resultados **5**

1. Introducción

En este laboratorio hemos diseñado y codificado un agente inteligente capaz de aprender a cocinar hamburguesas en una cocina virtual, inspirado en el famoso juego “Overcooked”. Nuestro agente, un pollito, aprende a navegar y completar las recetas utilizando **Aprendizaje por Refuerzo**.

Para lograrlo, hemos implementado el algoritmo **Q-Learning.** A través de un sistema de recompensas y penalizaciones, el agente va aprendiendo por sí mismo la secuencia correcta de acciones hasta convertirse en un cocinero eficiente.

2. Descripción del Entorno

El entorno es una cuadrícula de 8x6 celdas que simula una cocina con diferentes elementos:

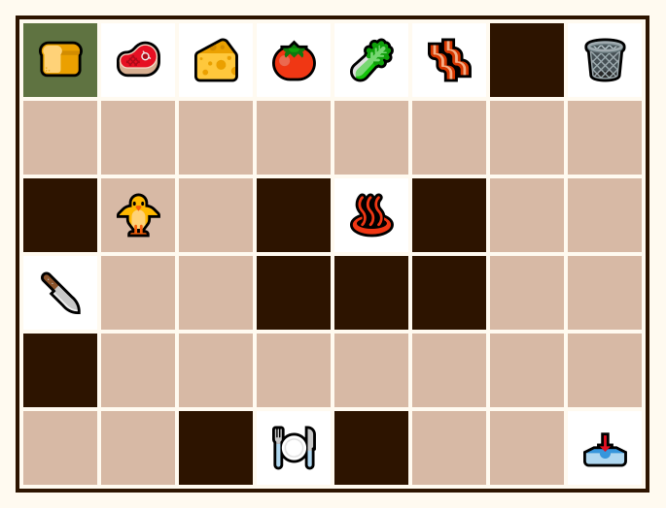


Figura 1. Mapa

2.1. Elementos

El mapa contiene celdas transitables, obstáculos y estaciones de trabajo interactivas:

- **Agente:** 🐥
- **Obstáculos:** Muebles que el agente debe aprender a esquivar (cuadrados oscuros).
- **Estaciones de Ingredientes:** Pan (🍞), Carne (🥩), Queso (🧀), Tomate (🍅), Lechuga (🥬) y Bacon (🥓).
- **Estaciones de Procesado:**
  - 🔪 **Cortar:** Para cortar el tomate, la lechuga y el pan.
  - ♨️ **Cocina:** Sartén para cocinar la carne y el bacon.
- **Otras Estaciones:**
  - 🍽️ **Emplatado:** Donde se ensambla la hamburguesa.
  - 📥 **Entrega:** Punto final donde se sirve el pedido.
  - 🗑️ **Basura:** Para desechar ítems incorrectos o quemados.

2.2. Dinámica de la Receta

El agente debe aprender a ejecutar las tareas en un orden específico para completar la hamburguesa correctamente:

1. 🍞 ➔ 🔪 ➔ 🍽️
2. 🥩 ➔ ♨️ ➔ 🍽️
3. 🧀 ➔ 🍽️
4. 🥓 ➔ ♨️ ➔ 🍽️
5. 🍅 ➔ 🔪 ➔ 🍽️
6. 🥬 ➔ 🔪 ➔ 🍽️
7. 🍔 ➔ 📥

Como se puede observar, cada ingrediente procesado tiene que ser llevado al plato antes de comenzar con el siguiente. Una vez completada la hamburguesa, el paso final es llevarla del plato a la estación de entrega. Como ayuda visual durante la ejecución, el mapa resalta en verde la siguiente casilla objetivo (ver *Figura 1*, donde el agente debe ir a por el pan).

Para evitar que el agente se pierda al inicio del entrenamiento, hemos diseñado un sistema de aprendizaje por niveles (**Curriculum Learning**). El pollito debe superar 5 niveles de dificultad incremental:

Nivel 1: Receta básica de Pan y Carne.
Siguientes niveles: Se añade un ingrediente extra por nivel (Queso, Bacon, Tomate, Lechuga).

El agente solo avanza de nivel cuando demuestra dominio del nivel actual (5 victorias consecutivas).

3. Definición del Problema

Para aplicar **Q-Learning**, hemos modelado el problema como un Proceso de Decisión de Markov (MDP):

3.1. Espacio de Acciones

El agente tiene **6 acciones discretas**:

1. Mover Arriba
2. Mover Abajo
3. Mover Izquierda
4. Mover Derecha
5. Interactuar: Usar la estación que tiene enfrente (coger, dejar, cortar, cocinar).
6. Esperar: Necesario mientras la comida se cocina en la sartén para evitar sacarla cruda.

3.2. Espacio de Estados

El estado no puede ser simplemente la posición del agente **(x, y)**, ya que la acción óptima depende del contexto (ej: estar frente a la mesa de cortar es bueno si tengo tomate, pero malo si tengo una hamburguesa terminada).

Por ello, el estado se define como una tupla compleja de 8 elementos:

- **(x, y):** Coordenadas del agente.
- **Paso Actual:** Índice de la receta que toca completar (0=Pan, 1=Carne...).
- **Mano:** Qué objeto lleva el agente (o None).
- **Sartén Ocupada:** (Binario 0/1).
- **Sartén Lista:** (Binario 0/1) ¿Ha pasado el tiempo mínimo de cocción?
- **Sartén Quemada:** (Binario 0/1) ¿Se ha excedido el tiempo máximo?
- **Error:** (Binario 0/1) Flag que indica si el agente lleva un objeto inútil para el paso actual, si es así, tendrá que llevarlo a la basura.
- **Nivel de Dificultad:** Nivel actual del curriculum.

4. Sistema de Recompensas

El diseño de la función de recompensa es crítico para guiar al agente. Hemos utilizado **Reward Shaping** para acelerar la convergencia y evitar que el agente vague sin rumbo.

4.1. Penalizaciones

- **Living Penalty:** -1 por cada paso (incentiva rutas cortas).
- **Choque con obstáculo:** -5 (incentiva evitar paredes).
- **Interacción vacía:** -0.5 (evita acciones repetitivas inútiles).
- **Tirar comida útil a la basura:** -50.
- **Tirar un plato lleno a la basura:** -500 (castigo severo para proteger el objetivo final).
- **Comida quemada o cruda:** -50 si se saca de la sartén incorrectamente.

4.2. Recompensas

- **Heurística de Distancia (Shaping):** +1 si el movimiento reduce la distancia Manhattan al objetivo actual.
- **Coger ingrediente correcto:** 0 (neutro, necesario para avanzar, ya que si le dábamos puntos se quedaba en un bucle cogiendo y tirando el ingrediente).
- **Cortar / Cocinar bien:** +30 (recompensa por sub-meta).
- **Emplatar ingrediente correcto:** +60 (recompensa por sub-meta crítica).
- **Coger plato lleno:** +50.
- **Entrega final:** +200 por completar una hamburguesa y Recompensa x2 al terminar el episodio completo.

4.3. Mecánica de Cocción

La sartén introduce una dinámica temporal que el agente debe aprender a gestionar:

- T < 2: Crudo (Penalización si se saca).
- 2 <= T <= 5: Cocinado Perfecto (Recompensa).
- T > 5: Quemado (El agente debe tirarlo a la basura y reiniciar ese ingrediente).

5. Implementación del Algoritmo Q-Learning

Se utiliza la ecuación de Bellman para actualizar iterativamente la tabla Q:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[R + \gamma \max Q(s', a') - Q(s,a)]$$

5.1. Hiperparámetros utilizados

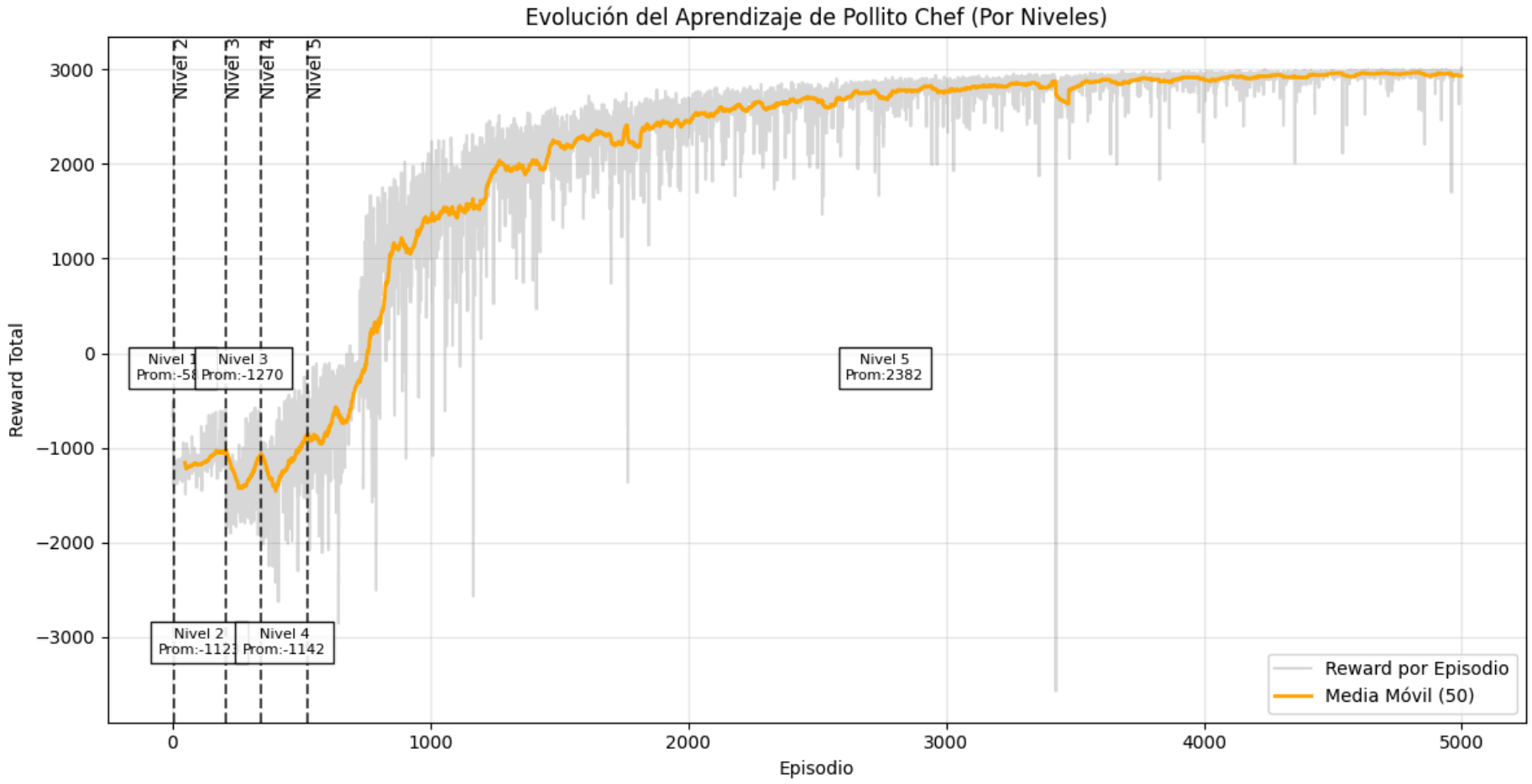
- **Tasa de aprendizaje (α):** 0.2. El agente actualiza sus conocimientos de forma moderada, equilibrando la estabilidad con la rapidez de aprendizaje.
- **Factor de descuento (γ):** 0.95. Se da una alta importancia a las recompensas futuras, fundamental para tareas largas como completar una receta.
- **Exploración (ε-greedy):**
    - Inicio: $\varepsilon = 1.0$ (100% aleatorio).
    - Decaimiento: $\varepsilon = \varepsilon \times 0.9995$ tras cada episodio.
    - Mínimo: $\varepsilon = 0.05$ (mantiene un 5% de exploración residual).

6. Visualización y Resultados

Como se puede ver en la *Figura 1*, hemos desarrollado un entorno visual con pygame para observar la evolución del agente. Dado que el entrenamiento consta de 5000 episodios, no se visualizan todos para agilizar el proceso. En su lugar, mostramos únicamente los 2 primeros episodios (para observar la ignorancia inicial del pollito), seguidos de un muestreo cada 500 episodios (por la terminal se muestra información cada 250 episodios). Finalmente, el último episodio lo mostramos más lento y con aleatoriedad 0 para verificar que el pollito ha aprendido la receta correctamente.

Cabe destacar que hemos limitado el número máximo de pasos permitidos por episodio. Esto es fundamental para evitar que el agente se quede atascado indefinidamente y para incentivarle a resolver la tarea en el menor tiempo posible.

El proceso concluye con la generación de una gráfica de rendimiento que muestra la evolución de las recompensas obtenidas. En la página a continuación se muestra cómo el agente mejora su puntuación progresivamente en una de las simulaciones.


Evolución del Aprendizaje de Pollito Chef (Por Niveles)
Nivel 2
Nivel 3
Nivel 4
Nivel 5
Nivel 1
Prom:-58
Nivel 3
Prom:-1270
Nivel 5
Prom:2382
Nivel 2
Prom:-1123
Nivel 4
Prom:-1142
Reward por Episodio
Media Móvil (50)
Reward Total
Episodio
3000
2000
1000
0
−1000
−2000
−3000
0
1000
2000
3000
4000
5000